

PROCESSOS & CICLO DE VIDA



iterar • entregar • aprender

Organizar o desenvolvimento em etapas, artefatos e ciclos de feedback para entregar software com previsibilidade.

- **Ciclo de vida:** requisitos → projeto → código → testes → manutenção
- **Cascata:** etapas sequenciais; mudanças tardias custam mais
- **Incremental:** entrega em partes funcionais
- **Iterativo:** refina a solução em ciclos
- **Ágil:** entregas frequentes + adaptação
- **Scrum:** Backlog, Sprint, Review e Retrospective

Dica: Processo não é burocracia por si só. O objetivo é reduzir risco, dar visibilidade e criar feedback rápido para corrigir o rumo cedo.

REQUISITOS, UML & MODELAGEM



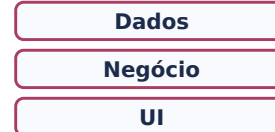
Entender o problema antes de codificar, registrando comportamento, regras e estrutura do sistema.

- **Requisito funcional:** o que o sistema deve fazer
- **Não funcional:** qualidade, desempenho, segurança, usabilidade etc.
- **História de usuário:** Como [papel], quero [objetivo], para [valor]
- **Critério de aceitação:** condição verificável para considerar algo pronto
- **Caso de uso:** interação ator ↔ sistema para atingir um objetivo
- **UML:** casos de uso, classes, sequência, atividade e estados

Dica: Requisito bom deve ser claro, testável e sem ambiguidade. Se não dá para verificar depois, provavelmente ainda está vago.

ARQUITETURA, DESIGN & QUALIDADE

camadas + interfaces



Estruturar o sistema para reduzir acoplamento, aumentar coesão e facilitar evolução e manutenção.

- **Coesão:** responsabilidades relacionadas ficam juntas
- **Acoplamento:** dependências entre módulos; quanto menor, melhor
- **SOLID:** princípios para design orientado a objetos sustentável
- **Camadas:** apresentação, negócio e dados separam responsabilidades
- **MVC:** Model, View e Controller
- **API:** contrato de comunicação entre componentes/sistemas
- **Padrões:** soluções recorrentes como Factory, Strategy e...

Dica: Prefira módulos pequenos, coesos e com interfaces claras. Alta coesão + baixo acoplamento facilita teste, troca e evolução.

TESTES, VERSIONAMENTO & MANUTENÇÃO



Controlar mudanças e detectar defeitos cedo por automação, integração contínua e disciplina de manutenção.

- **Unitário:** verifica uma unidade isolada
- **Integração:** verifica interação entre módulos
- **Sistema:** valida o produto completo
- **Regressão:** verifica se mudanças quebraram algo
- **Pirâmide:** muitos unitários; poucos E2E
- **Git:** commit, branch, merge e pull request
- **CI/CD:** build, testes e entrega automatizados

Dica: Automatize o que é repetitivo. Testes e versionamento não eliminam defeitos, mas tornam mudanças mais seguras e rastreáveis.